

7.1 Execução: types, helpers e regras de cascata — status de entrega

Bloco 7.1 Execução: types, helpers e regras de cascata · Sprint 7 — Execução: estrutura e apontamento · Data: 2026-09-23 · Branch: `main` · Ambiente: `gc-dev` (`gzbmhgnpoehoirmnidmgg`)

Resumo

Implementado e validado no fechamento da sprint (rodada completa de 2026-09-24, detalhe no `7.6-status-entrega.md`). Os 12 casos novos de `execucao.test.ts` passaram dentro dos 255 da suíte. Não há migration nem tela: só tipos, helpers puros e testes. A tela `/execucao` continua como "Em construção" até o 7.2.

1. As cinco atividades do bloco

ATIVIDADE DO CLICKUP	ONDE FICOU
Execucao/ExecucaoInsert/ExecucaoUpdate e tipos derivados	<code>src/lib/types.ts</code> . <code>Execucao</code> estreita os quatro <code>*_status</code> para <code>EtapaStatus</code> (<code>pendente</code> , <code>andamento</code> ou <code>concluido</code>), como <code>Contrato</code> faz com <code>status</code> . Novo <code>ExecucaoListItem</code> , a execução com o item, para a listagem do 7.2
<code>src/lib/execucao.ts</code> : <code>validarCascata(fab, ent, inst, med, total)</code>	Com a assinatura do enunciado. Confere número válido, sem negativo e até 3 casas (a precisão de <code>numeric(10,3)</code>), e depois a cascata na ordem das etapas, parando no primeiro bloqueio. Devolve a etapa e a mensagem, por exemplo "Só é possível entregar 12 porque só 12 foram fabricados", com a concordância de nenhum, um e vários
Helper do máximo permitido por etapa	<code>limitesDaEtapa(etapa, qtds, total)</code> devolve <code>{ min, max }</code> . O máximo é a etapa anterior (o total, na fabricação). O mínimo é a etapa seguinte: baixar a fabricação para menos do que já foi entregue quebra o CHECK da entrega. <code>maximoDaEtapa</code> é o atalho "concluir etapa" do 7.3
Progresso percentual por etapa e geral	<code>progressoEtapa(qtd, total)</code> , <code>progressoGeral(qtds, total)</code> (média das quatro etapas) e <code>progressoAgregado(execucoes)</code> , que é o totalizador da obra no 7.2: soma das quantidades sobre a soma dos totais

ATIVIDADE DO CLICKUP	ONDE FICOU
Labels e cores de status	<code>STATUS_ETAPA_LABELS</code> (Pendente, Em andamento, Concluído), <code>STATUS_ETAPA_CORES</code> (classes do selo e da barra), <code>ETAPA_LABELS</code> (Fabricação, Entrega, Instalação, Medição) e <code>statusDaEtapa</code> , que espelha o <code>GENERATED</code> para a prévia na tela

E ainda `mensagemDeErroExecucao`, que traduz os CHECKs da cascata (`execucao_*_qtd_check`), o sequencial repetido, a FK do item e o trigger de `valor_unit`.

2. Decisões

- **O banco continua sendo a regra.** A cascata são quatro CHECKs de coluna na tabela `execucao`. O status de cada etapa é `GENERATED`, e as datas de início, atualização e fim vêm do trigger `execucao_preenche_datas`. Os helpers só repetem a regra para a tela limitar o input e explicar o bloqueio antes do POST, e para a action recusar com mensagem legível. Nenhuma data nem status é gravado pela aplicação.
- **Comparação em milésimos**, e não com `<=` em ponto flutuante: `0,1 + 0,2` tem de ser igual a `0,3`, como no `numeric` do banco.
- **Progresso geral é a média das quatro etapas**, cada uma com o mesmo peso. A view `obras_com_valores` mede o progresso da obra só pela medição (`progresso_itens_pct`). As duas medidas respondem a perguntas diferentes, e o 7.2 mostra as quatro etapas separadas.

3. Achado para o 7.4: `quantidade_total` com várias execuções por item

A migration `20260511151924` (a "004") tirou o `unique(item_id)` e criou o `sequencial`, mas os dois triggers de quantidade continuaram escritos para uma execução por item:

- `execucao_set_quantidade_total` (BEFORE INSERT) **sobrescreve** `quantidade_total` com a quantidade inteira do item, mesmo que o insert mande outra. Uma segunda execução do mesmo item nasce com o total do item, e não com a parte dela.
- `itens_sincroniza_execucao_qtd` (quando `itens.quantidade` muda) grava a quantidade do item em **todas** as execuções dele.
- Ao reduzir a quantidade de um item para menos do que uma execução já fabricou, o `fab_qtd <= quantidade_total` recusa. O update do item falha com o erro cru do CHECK.

Com isso, a atividade do 7.4 "soma das quantidades não ultrapassa a do item" não tem como ser cumprida. **O 7.4 precisa de uma migration** que faça o insert respeitar o `quantidade_total` enviado (e só usar a do item quando vier nulo) e que troque a sincronização por outra regra. Proponho:

- com uma execução, a sincronização continua como hoje;
- com várias, a mudança de quantidade do item não reparte sozinha, e a tela pede para ajustar.

É decisão de modelagem, e fica para o início do 7.4. **Atualização:** resolvido no 7.4, com a migration `20260924110000_execucao_varias_por_item.sql`.

4. Plano de validação: rodado no fechamento da sprint

CAMADA	ESTADO	O QUE ENTROU
unitário	passou (255/255 casos da suíte)	12 casos em <code>src/lib/execucao.test.ts</code> : ordem das etapas, espelho do GENERATED (inclusive decimal), cascata válida, os quatro bloqueios com a mensagem exata, a concordância de nenhum e de um, decimal com vírgula e mais de 3 casas, negativo e NaN na etapa certa, <code>limitesDaEtapa</code> , "concluir etapa", progresso por etapa, geral e agregado, e a tradução dos erros do banco
estático, build, runtime, dados, escrita, navegador	passou (limpo, 42 rotas, 137/137, 35/35, 334/334, 135/135)	nada novo do bloco: não tem tela, query nem action. Os 4 CHECKs que <code>mensagemDeErroExecucao</code> traduz só ganharam o nome esperado no fechamento (migration <code>20260924120000</code> , ver 7.6)

5. Pendências

- Nenhuma. O plano rodou e passou no fechamento da sprint.
- A migration de `quantidade_total` (seção 3) foi feita no 7.4 (`20260924110000`).
- A rodada de fechamento achou que os CHECKs traduzidos por `mensagemDeErroExecucao` tinham nome automático no banco; corrigido pela migration `20260924120000` (ver 7.6, seção 2.2).

6. Registro para quem for operar prod

Nenhuma migration.